

1. What is Zustand ?

- Zustand is a small, fast and scalable state management library for React.
- It uses a simple store based approach.
- No Providers, No Complex Setup.
- Combines simplicity of useState with power of Redux.

Small
Fast
Simple



2. Core Concepts

- **Store** → Central place to hold state.
- **useStore()** → Hook to access store.
- **get()** → Read state outside components.
- **set()** → Update state.
- **Action** → Functions that update state.



3. Basic Store Example

```
import { create } from 'zustand'
// create a store
const useCounterStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
  decrement: () => set((state) => ({ count: state.count - 1 })),
  reset: () => set({ count: 0 })
}))
```

Note:

- **set()** merges state by default.
- You can also return an object to update.

Explanation

- **count** → initial state
- **increment()** → increases count
- **decrement()** → decreases count
- **reset()** → sets count to 0

Looks like useState,
Works like Redux !

4. Using Store in Components

```
import React from 'react'
import useCounterStore from './store/counterStore'

function Counter() {
  const count = useCounterStore((state) => state.count)
  const { increment, decrement, reset } = useCounterStore()
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={increment}>+1</button>
      <button onClick={decrement}>-1</button>
      <button onClick={reset}>Reset</button>
    </div>
  )
}
```

useStore()
subscribes only
to the used
state (Optimized
re-renders).

5. Advanced: Using get()

// get() helps to read current state inside actions

```
const useUserStore = create((set, get) => ({
  name: 'Kapil',
  updateName: (newName) => set({ name: newName }),
  capitalizeName: () => {
    const current = get().name
    set({ name: current.toUpperCase() })
  }
}))
```

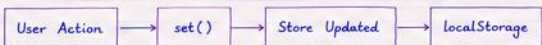
get()
reads current
state inside
actions.

6. Persist Middleware (State Persistence)

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

const usePersistStore = create(
  persist(
    (set) => ({
      count: 0,
      increment: () => set((state) => ({ count: state.count + 1 })),
    }),
    { name: 'my-store' } // store name in localStorage
  )
)
```

State will
be saved in
localStorage
automatically.



(persist saves)

7. Devtools Middleware (Redux DevTools Support)

```
import { devtools } from 'zustand/middleware'

const useDevtoolsStore = create(
  devtools((set) => ({
    count: 0,
    increment: () => set((state) => ({ count: state.count + 1 })),
  })),
)
```

Check state
changes in
Redux
DevTools.

8. Combining Middlewares

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { devtools } from 'zustand/middleware'

const useStore = create(
  devtools(
    persist(
      (set) => ({
        count: 0,
        increment: () => set((state) => ({ count: state.count + 1 })),
      }),
      { name: 'my-store' }
    )
  )
)
```

Order
devtools(
 persist(
 yourStore
)
)

Outer → devtools
Inner → persist

9. Immutable Updates

- Zustand allows mutable style updates.
- But always return new object (best practice).

Always
return new
object / array.

```
const useTodoStore = create((set) => ({
  todos: [],
  addTodo: (text) => set((state) => ({
    todos: [...state.todos, { id: Date.now(), text: text, done: false }]
  })),
  toggleTodo: (id) => set((state) => ({
    todos: state.todos.map(t) =>
      t.id === id ? {...t, done: !t.done} : t
  })
})),
}))
```

10. Subscribing Outside Components

```
// Useful for non-React logic
const unsub = useCounterStore.subscribe((state) =>
  console.log('Count changed.', state.count)
)

// later call to stop
unsub()
```

subscribe()
to listen
state changes
outside UI.

11. Advantages of Zustand

- ✓ Very small size (~1KB)
- ✓ Simple API
- ✓ No Providers
- ✓ Fast performance
- ✓ Works well with TypeScript
- ✓ DevTools support
- ✓ Middlewares for advanced needs

12. When to Use Zustand ?

- Small to medium applications.
- When you want simple and lightweight state management.
- When Redux feels heavy.
- When you want good performance + less boilerplate.

QUICK REVISION

Read State

```
const value = useStore(
  (state) => state.value
)
```

Update State

```
set((state) => ({
  value: newValue
})))
```

Outside Component

```
const value = useStore.getState()
```

Subscribe

```
const unsub = useStore.subscribe(
  (state) => console.log(state)
)

unsub() // stop
```

Flow

Action



set()



Store Update



Subscriber
(Component)
re-renders

★ Zustand = Simplicity + Performance + Power ★